

Compute, Data & Parameters.

Three quantities govern a language model’s loss — parameters N , training tokens D , and compute C — and they are not independent: $C \approx 6ND$. This note derives the factor of 6 from first principles, shows how a fixed compute budget forces a trade-off between N and D , and contrasts Kaplan’s “scale the model” verdict with the Chinchilla correction.

THE EQUATION

$$C \approx 6ND \quad (\text{training FLOPs})$$

Worked example. $N = 1.5 \times 10^8$, $D = 3 \times 10^8 \Rightarrow C = 2.7 \times 10^{17}$; budgets $C = 10^{19}, 10^{21}, 10^{23}$ split into (N, D) .

Topic 2 of 2 in this module · companion to the course page · v1.0

Contents

1	Three knobs, one loss	2
2	Deriving the factor 6	3
3	The fixed-budget trade-off	3
4	Kaplan's allocation, computed	4
5	Chinchilla foreshadow	4
6	Properties that matter	5
7	Where this sits in the track	5
A	Reproducible (NumPy)	5

1 Three knobs, one loss

Topic 1 gave three power laws, one per resource: parameters N , training tokens D , and compute C . In practice you do not choose all three freely — they are tied together by how much arithmetic a transition over the data costs.

$$C \approx 6 N D \quad (1)$$

Symbol	Meaning	Units
N	non-embedding parameters	count
D	training tokens seen (one epoch)	tokens
C	total training compute	FLOPs
6	FLOPs per parameter per token	dimensionless

Each parameter touches each token a small, *fixed* number of times during training. So total work is just (work per parameter-token) $\times$ (parameters) $\times$ (tokens). That constant is 6. Everything else — which architecture, how deep — washes out into N .

2 Deriving the factor 6

A dense matrix multiply that uses P parameters costs about $2P$ FLOPs *per token*: each weight contributes one multiply and one add (a multiply-add is two floating-point operations). So the **forward** pass over one token is

$$\text{forward} \approx 2N \text{ FLOPs/token.}$$

The **backward** pass computes two sets of gradients — with respect to the activations (to propagate error to the previous layer) and with respect to the weights — each of which costs about as much as the forward pass. So backward $\approx 2 \times (2N) = 4N$ FLOPs/token. Adding them:

$$\underbrace{2N}_{\text{fwd}} + \underbrace{4N}_{\text{bwd}} = 6N \text{ FLOPs/token.}$$

Multiplying by D tokens gives Eq. (1):

$$C = (2N + 4N) \times D = 6ND. \quad (2)$$

Pass	FLOPs / token	Why
Forward	$2N$	one multiply + one add per weight
Backward (activations)	$2N$	same matmul shape, transposed
Backward (weights)	$2N$	gradient of loss w.r.t. each weight
Total	$6N$	forward + backward

Concrete arithmetic. Take $N = 1.5 \times 10^8$ parameters and $D = 3 \times 10^8$ tokens:

$$6N = 6 \times 1.5 \times 10^8 = 9.0 \times 10^8, \quad C = (9.0 \times 10^8) \times (3 \times 10^8) = 2.7 \times 10^{17} \text{ FLOPs.}$$

3 The fixed-budget trade-off

Rearranging Eq. (1), C pins the *product* ND , so along a budget line $D = C/(6N)$: spend more on parameters and you must train on fewer tokens, and vice versa. On a log-log plot the budget line is straight with slope -1 :

$$\log D = \log \frac{C}{6} - \log N.$$

Scaling laws then say each point on that line has a predictable loss, and *one* point on it — the optimal split (N^*, D^*) — minimises the loss for that budget.

Loss is bottlenecked by whichever of N or D is undersized. A huge model on a tiny dataset memorises and is data-starved; a tiny model on oceans of data underfits and is parameter-starved. The optimum balances the two so neither is the lone bottleneck — like a bucket whose level is set by its shortest stave.

4 Kaplan's allocation, computed

Kaplan et al. found the optimal split grows the model fast: roughly $N^* \propto C^{0.73}$ and $D^* \propto C^{0.27}$ (the exponents sum to 1, consistent with $C = 6ND$). Using $D^* = k_D C^{0.27}$ with $k_D \approx 4.28 \times 10^4$ and then $N^* = C/(6D^*)$:

C (FLOPs)	N^* (params)	D^* (tokens)	D^*/N^*	check $6N^*D^*$
10^{19}	2.889×10^8	5.768×10^9	19.96	1.0×10^{19}
10^{21}	8.333×10^9	2.000×10^{10}	2.40	1.0×10^{21}
10^{23}	2.403×10^{11}	6.935×10^{10}	0.29	1.0×10^{23}

Read the slopes off the table: across the $10^4 \times$ span of C , N^* grows by $10^{2.92}$ and D^* by only $10^{1.08}$, i.e. exponents 0.73 and 0.27. The striking column is D^*/N^* : under the Kaplan rule it *falls* as compute grows — the recipe pours budget into parameters and starves the data.

The allocation as a heatmap. Share of budget's growth captured by N (indigo) vs. D (lighter) at each scale — N dominates and widens its lead:

	$C=10^{19}$	$C=10^{21}$	$C=10^{23}$
$\log_{10} N^*$	8.46	9.92	11.38
$\log_{10} D^*$	9.76	10.30	10.84

Under a fixed budget $C = 6ND$, you are choosing a *point on a line*, not two free numbers. Kaplan's exponents (~ 0.73 for N , ~ 0.27 for D) say: when compute grows, make the model much bigger and the dataset only a little bigger. This is the reasoning that justified a 175B-parameter GPT-3.

5 Chinchilla foreshadow

The Kaplan rule's falling D^*/N^* is a warning sign — it implies that at GPT-3 scale the models were *undertrained* (too few tokens for their size). **Chinchilla** (Hoffmann et al., 2022; covered in Module 2.4) re-fit the laws with learning-rate schedules matched to each run and found N and D should scale *together*, roughly $D^* \approx 20 N^*$ tokens per parameter. Under $C = 6ND$ with $D = 20N$:

$$C = 6N(20N) = 120 N^2 \Rightarrow N^* = \sqrt{C/120}, \quad D^* = 20N^*.$$

C (FLOPs)	N^* (Chinchilla)	D^* (Chinchilla)	D^*/N^*
10^{19}	2.887×10^8	5.774×10^9	20.0
10^{21}	2.887×10^9	5.774×10^{10}	20.0
10^{23}	2.887×10^{10}	5.774×10^{11}	20.0

Same budget, very different split: at $C = 10^{23}$ Chinchilla prescribes a model $\sim 8 \times$ *smaller* than Kaplan's but trained on $\sim 8 \times$ more tokens, holding D/N fixed at 20 instead of letting it collapse.

6 Properties that matter

1. **Coupling.** $C \approx 6ND$ means any two of $\{C, N, D\}$ determine the third.
2. **Budget line.** Fixed C is a slope- -1 line in $(\log N, \log D)$; the optimum is a single point on it.
3. **Exponent sum.** $N^* \propto C^a$, $D^* \propto C^{1-a}$ with $a + (1-a) = 1$, forced by the product constraint.
4. **Bottleneck.** Loss is set by the scarcer of N and D ; doubling the abundant one barely helps.

The factor 6 counts only the dense matmul FLOPs and treats attention's quadratic term as negligible. For long context lengths L relative to model width, attention adds an $\mathcal{O}(L)$ correction and $C = 6ND$ under-counts. Likewise the Kaplan exponents were fit *without* fully tuned learning-rate schedules; trusting them at extreme scale is exactly the error Chinchilla corrected. Treat $6ND$ as a leading-order estimate, not gospel.

7 Where this sits in the track

Topic 1 gave the single-variable law $L(N)$; this topic coupled N , D , C and posed the allocation problem. Module 2.3 uses these models (GPT-3 scale) to study *in-context learning*; Module 2.4 returns with Chinchilla's compute-optimal correction in full.

A Reproducible (NumPy)

Inputs: $C = 6ND$; Kaplan $D^* = k_D C^{0.27}$, $k_D = 4.28 \times 10^4$; Chinchilla $D^* = 20N^*$.

```
import numpy as np
# factor-6 check
N, D = 1.5e8, 3e8
print("C =", 6*N*D)                # C = 2.7e+17

kD = 2.0e10 / (1e21**0.27)          # anchor: D*=2e10 at C=1e21
for C in [1e19, 1e21, 1e23]:
    Dk = kD * C**0.27
    Nk = C / (6*Dk)
    print(f"Kap C={C:.0e} N={Nk:.2e} D={Dk:.2e} D/N={Dk/Nk:.2f}")
# Kap C=1e+19 N=2.89e+08 D=5.77e+09 D/N=19.96
# Kap C=1e+21 N=8.33e+09 D=2.00e+10 D/N=2.40
# Kap C=1e+23 N=2.40e+11 D=6.94e+10 D/N=0.29

for C in [1e19, 1e21, 1e23]:
    Nc1 = np.sqrt(C/120); Dc1 = 20*Nc1          # Chinchilla 20 tokens/param
    print(f"Chi C={C:.0e} N={Nc1:.2e} D={Dc1:.2e} D/N={Dc1/Nc1:.1f}")
# Chi C=1e+19 N=2.89e+08 D=5.77e+09 D/N=20.0
# Chi C=1e+21 N=2.89e+09 D=5.77e+10 D/N=20.0
# Chi C=1e+23 N=2.89e+10 D=5.77e+11 D/N=20.0
```

